

# LocaLLM Docs: Sistema Local de Análise Documental com Retrieval-Augmented Generation

Eduardo Brito, M15384

Universidade da Beira Interior, Covilhã, Portugal

## Abstract

This work describes the development of **LocaLLM Docs**, a local document analysis system based on the *Retrieval-Augmented Generation* (RAG) paradigm. The system allows users to upload documents in PDF and Word formats, interact with their content through a natural language chat, generate executive summaries, extract keywords, and export the results to a PDF report. The entire pipeline operates in *offline* mode, without relying on external APIs, utilizing Ollama with the `llama3.2:3b` model and `nomic-embed-text` embeddings, ChromaDB as the local vector database, and the `Helsinki-NLP/opus-mt-en-pt` model for automatic English→Portuguese translation. The interface was implemented using Streamlit, and the code is organized into a modular architecture comprising nine Python files. Tests conducted with academic documents in Portuguese and English demonstrate that the system produces accurate factual responses, coherent summaries, and relevant keywords, with response times of under 15 seconds on an NVIDIA RTX 3060 Laptop GPU.

## Keywords

Retrieval-Augmented Generation, Natural Language Processing, Local Language Models, Ollama, ChromaDB, Streamlit

## 1. Introdução

Hoje em dia, usar a linguagem natural para "conversar" com documentos é uma das coisas mais úteis e diretas dos modelos de linguagem (LLMs). Tarefas como fazer investigação académica, rever relatórios empresariais ou ler documentação técnica obrigam-nos a lidar com imenso texto desestruturado. Ter um sistema capaz de pesquisar e resumir essa informação de forma automática poupa-nos imenso tempo e esforço mental [1].

O grande problema é que a grande maioria das ferramentas atuais depende de APIs externas (como as da OpenAI ou da Anthropic). Isto levanta três questões complicadas: a falta de privacidade, os custos de utilização e a obrigatoriedade de ter ligação à Internet. Para um investigador que lida com dados sensíveis, ou simplesmente para um estudante que quer analisar os seus apontamentos sem enviar os seus PDFs para os servidores de outras empresas, estas limitações são um obstáculo gigante.

Foi exatamente para dar resposta a esta necessidade que desenvolvi o LocaLLM Docs. Trata-se de um sistema completo de análise documental que corre a 100% no computador do utilizador, sem fazer qualquer chamada a serviços externos. O próprio nome reflete essa ideia: junta a execução estritamente Local, o uso de LLMs e o foco em Documentos. Na prática, a aplicação consegue ler ficheiros PDF e Word, criar um índice vetorial na máquina e deixar o utilizador fazer perguntas ao texto como se estivesse num chat. Além disso, gera resumos à medida, extrai as palavras-chave mais importantes e permite exportar toda a análise para um relatório (PDF) bem estruturado.

Resumindo, o objetivo principal deste trabalho é demonstrar que é perfeitamente possível construir um sistema RAG (Retrieval-Augmented Generation) de excelência, totalmente funcional e completo, utilizando apenas ferramentas open-source configuradas localmente.

## 2. Arquitetura do Sistema

### 2.1. Visão Geral Modular

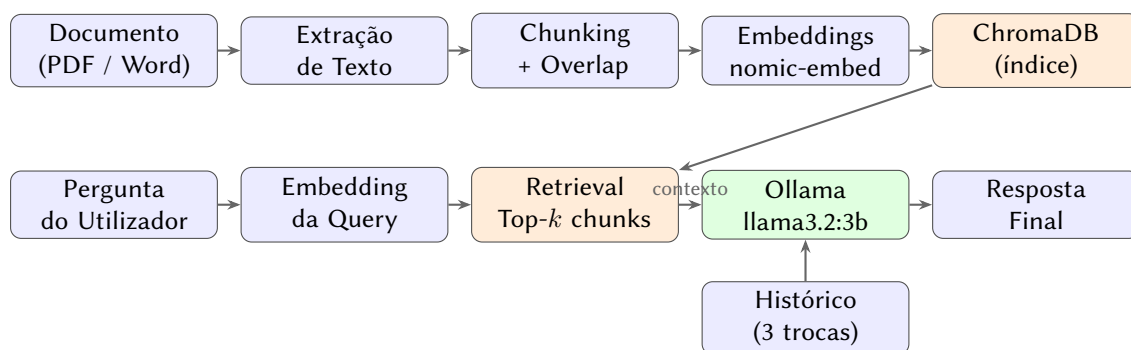
Para manter o código organizado e fácil de perceber, dividi a arquitetura do projeto em nove ficheiros Python. Mantive tudo na mesma pasta (uma estrutura plana) para simplificar a navegação, mas fpo atribuído a cada ficheiro uma responsabilidade muito específica:

- **app.py** — É o motor principal da aplicação. Corre a interface em Streamlit, gere a barra lateral de configurações e organiza as funcionalidades em quatro separadores práticos.
- **models.py** — É onde centralizei o carregamento de todos os modelos. Usei o decorador `@st.cache_resource` para garantir que ficam retidos na memória e não têm de ser recarregados do zero a cada clique.
- **document.py** — Trata do trabalho "sujo" de ler os PDFs (via PyMuPDF) e os ficheiros Word (via python-docx), fazendo logo a divisão do texto em fragmentos (*chunks*) com a respetiva sobreposição (*overlap*).
- **vectorstore.py** — Fica encarregue de criar e pesquisar no nosso índice vetorial ChromaDB, que corre de forma super rápida diretamente na memória.
- **rag.py** — É onde acontece a pipeline RAG. Vai buscar o contexto mais relevante ao documento e gere o histórico de mensagens para o modelo não se perder.
- **summarizer.py** — Responsável por gerar os resumos. Incluí aqui uma função que limpa automaticamente as referências bibliográficas do texto original, para evitar que o modelo se confunda e as inclua no resumo.
- **keywords.py** — Força o modelo de linguagem a extrair as palavras-chave e a devolvê-las no formato exato de uma lista JSON estruturada.
- **translator.py** — Lida com a tradução automática de Inglês para Português usando o modelo MarianMT, processando o texto em pequenos lotes (*batches*) para não sobrecarregar o sistema.
- **exporter.py** — Agarra em todos os resultados obtidos e desenha o relatório final em PDF utilizando a biblioteca fpdf2.

Ao separar o sistema em módulos completamente independentes, a grande vantagem prática é que é possível testar, afinar ou substituir uma peça específica (como o módulo de tradução, por exemplo) de forma isolada, sem correr o risco de estragar o resto da aplicação.

### 2.2. Pipeline de Processamento

A Figura 1 mostra como funciona todo o fluxo de dados do sistema, desde o momento em que o utilizador carrega o ficheiro até à geração da resposta final no chat.



**Figure 1:** Pipeline de processamento do LocaLLM Docs.

Assim que o utilizador faz o upload do documento, a aplicação arranca logo com a **fase de indexação**, que se divide em três passos principais:

1. **Extração de texto:** O sistema lê o ficheiro submetido e extrai todo o texto limpo, usando o PyMuPDF para ficheiros PDF ou o python-docx para documentos Word, garantindo que a ordem original dos parágrafos se mantém.
2. **Chunking com sobreposição (overlap):** Como o texto completo seria demasiado grande para o modelo processar de uma só vez, dividimo-lo em pequenos fragmentos (*chunks*) com um tamanho padrão de 500 palavras. Foi também definido um *overlap* de 80 palavras entre blocos vizinhos. Esta margem é fundamental: se uma frase longa for cortada mesmo na fronteira entre dois blocos, ela acaba por aparecer em ambos. Isto preserva o sentido do texto e ajuda o sistema a encontrar a informação correta mais tarde.
3. **Indexação vetorial:** Para terminar esta fase, cada um desses blocos é convertido num vetor matemático através do modelo de *embeddings* `omic-embed-text` e guardado numa coleção do ChromaDB criada diretamente na memória RAM.

Com o documento bem indexado na base de dados, entramos na **fase de consulta**. Sempre que o utilizador faz uma pergunta, o sistema executa o seguinte fluxo:

1. A pergunta é convertida num vetor de *embeddings* usando exatamente o mesmo modelo (`omic-embed-text`).
2. O ChromaDB faz uma pesquisa rápida por similaridade de cosseno e devolve os  $k$  blocos de texto do documento que são mais parecidos e relevantes para responder àquela dúvida.
3. Pegamos nesses blocos de contexto recuperados, na pergunta atual e nas últimas mensagens do histórico (de forma resumida) e juntamos tudo num único *prompt* bem estruturado que é enviado para o LLM local.
4. O modelo (servido pelo Ollama) gera a resposta final em português e apresenta-a na interface, indicando também ao utilizador quais foram os fragmentos exatos do texto original que serviram de base para a resposta.

### 2.3. Justificação dos Hiperparâmetros

A escolha dos hiperparâmetros baseou-se em vários testes práticos até encontrar o equilíbrio ideal para o sistema:

- **Tamanho do chunk (500 palavras):** Pareceu o melhor ponto entre não perder o contexto e não sobrecarregar o sistema. Se os blocos fossem muito pequenos (menos de 100 palavras), o modelo perdia a noção da ideia geral do texto. Se fossem demasiado grandes (mais de 1000 palavras), a pesquisa por similaridade começava a perder o foco e aproximávamo-nos perigosamente do limite de contexto do modelo de *embeddings*.
- **Overlap (80 palavras):** Isto corresponde a cerca de 16% do tamanho de cada bloco. Na prática, reparei que é a margem de segurança perfeita para garantir que uma frase mais longa, que acabe cortada a meio de dois blocos, não perca o seu sentido original. Ao mesmo tempo, evitamos duplicar texto de forma exagerada na base de dados.
- **$k = 6$  chunks:** Quando se começou a testar o sistema, era usado  $k = 4$  (ou seja, eram recuperados apenas os 4 blocos mais parecidos com a pergunta). No entanto, o modelo dava respostas incompletas sempre que a informação necessária estava espalhada por várias secções do documento. Ao aumentar este valor para 6, a precisão das respostas melhorou imenso e o tempo de espera manteve-se praticamente igual.
- **Histórico limitado a 200 caracteres:** Durante as primeiras interações de teste, ao enviar as respostas longas do assistente integralmente de volta no *prompt*, o modelo ficava "contaminado". Basicamente, o LLM começava a repetir o que já tinha dito no passado em vez de responder à minha nova pergunta. Ao cortar o tamanho das mensagens antigas aos 200 caracteres, esse problema foi resolvido: o sistema percebe o contexto da conversa, mas foca-se totalmente naquilo que lhe estou a perguntar no momento.

### 3. Recursos de Linguagem Natural Incorporados

#### 3.1. Modelo de Linguagem: Ollama + llama3.2:3b

O modelo de linguagem principal é o llama3.2:3b, servido localmente através do Ollama [2]. Este modelo, com 3 mil milhões de parâmetros quantizados para 4-bit, foi escolhido por equilibrar qualidade de respostas e requisitos de memória GPU, cabendo inteiramente nos 6GB VRAM da NVIDIA RTX 3060 Laptop GPU utilizada. A integração é feita via langchain-ollama, que expõe o modelo como um objeto compatível com a interface LangChain, permitindo trocar o modelo base sem alterar o restante código.

A temperatura foi configurada para 0.2, reduzindo a variabilidade das respostas e promovendo maior fidelidade ao contexto fornecido, comportamento desejável num sistema de análise documental onde a precisão factual é prioritária.

#### 3.2. Embeddings: nomic-embed-text

Os embeddings semânticos são gerados pelo modelo nomic-embed-text [3], também servido via Ollama. Este modelo produz vetores de dimensão 768, capturando o significado semântico dos chunks para além da simples correspondência de palavras-chave. A escolha de um modelo de embeddings dedicado (em vez de usar o próprio LLM para gerar embeddings) é uma prática padrão em sistemas RAG, pois modelos de embeddings são otimizados especificamente para similaridade semântica.

#### 3.3. Base de Dados Vetorial: ChromaDB

O armazenamento e pesquisa vetorial é feito pelo ChromaDB [4], configurado em modo *in-memory* sem servidor externo. Esta configuração elimina a necessidade de instalar e gerir um serviço separado, simplificando o setup do sistema. A pesquisa de similaridade por cosseno permite recuperar os  $k$  chunks mais relevantes para cada consulta.

A Tabela 1 resume o stack tecnológico completo do sistema.

**Table 1**

Stack tecnológico do LocaLLM Docs

| Componente     | Tecnologia                   | Versão              |
|----------------|------------------------------|---------------------|
| Interface      | Streamlit                    | $\geq 1.35$         |
| LLM            | Ollama + llama3.2:3b         | 3B params           |
| Embeddings     | nomic-embed-text             | 768 dim             |
| Vector store   | ChromaDB                     | $\geq 0.5$          |
| Extração PDF   | PyMuPDF                      | $\geq 1.24$         |
| Extração Word  | python-docx                  | $\geq 1.1$          |
| RAG framework  | LangChain + langchain-ollama | $\geq 0.2$          |
| Tradução       | Helsinki-NLP/opus-mt-en-pt   | $\sim 300\text{MB}$ |
| Exportação PDF | fpdf2                        | $\geq 2.7$          |

#### 3.4. Tradução Automática: Helsinki-NLP/opus-mt-en-pt

O módulo de tradução utiliza o modelo opus-mt-en-pt [5] da família MarianMT, disponível no Hugging Face. Este modelo, com aproximadamente 300MB, é descarregado uma única vez e fica em cache local para uso completamente offline. A tradução é aplicada em batch sobre frases segmentadas por pontuação, respeitando o limite máximo de 512 tokens por sequência de entrada. Este módulo funciona como camada de segurança: se o LLM gerar uma resposta em inglês, o que pode ocorrer com documentos muito técnicos em inglês ou com modelos diferentes, a resposta é automaticamente traduzida para português antes de ser apresentada ao utilizador.

### 3.5. Exportação PDF: fpdf2

A geração do relatório PDF é feita com a biblioteca fpdf2 [6], sem dependências externas. O relatório é composto por três secções: o resumo executivo gerado, a lista de palavras-chave, e o histórico completo de chat da sessão com distinção visual entre mensagens do utilizador e do assistente.

## 4. Funcionalidades Implementadas

### 4.1. Chat RAG com Histórico de Conversa

O sistema implementa uma pipeline RAG completa descrita na Secção 2.2. O prompt enviado ao LLM inclui três componentes: o contexto recuperado do documento, o histórico das últimas três trocas conversacionais (com respostas do assistente limitadas a 200 caracteres), e a pergunta atual do utilizador. O prompt instrui explicitamente o modelo a responder em português europeu, a não inventar informação que não esteja no contexto fornecido, e a não exceder três parágrafos por resposta.

A Tabela 2 apresenta exemplos reais de interações obtidas durante os testes com o artigo académico sobre o NotebookLM.

**Table 2**

Exemplos de interações reais do sistema com o documento de teste (artigo sobre NotebookLM, 8 páginas)

| Pergunta do Utilizador                                                   | Resposta do LocalLLM Docs (resumida)                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Qual é o modelo de linguagem que o NotebookLM utiliza?                   | O NotebookLM utiliza o modelo de linguagem Gemini 1.5 Pro.                                                                                                                                                                                                                 |
| Quais são as duas inovações estruturais do Gemini 1.5 Pro?               | As duas inovações são: (1) a arquitetura Mixture-of-Experts (MoE), que ativa apenas os subconjuntos de redes neuronais mais relevantes para cada token; (2) a janela de contexto expandida para 2 milhões de tokens, que resolve o fenómeno <i>Lost in the Middle</i> .    |
| Qual é o preço de subscrição do NotebookLM?                              | Não encontrei informação suficiente no documento para responder a essa pergunta.                                                                                                                                                                                           |
| O que é o source grounding? (seguido de) E quais são as suas limitações? | [2ª pergunta, sem repetir contexto] O NotebookLM apresenta as seguintes limitações: qualidade da fonte ( <i>Garbage In, Garbage Out</i> ), limites de processamento físico (50 fontes por caderno), e comportamento “confiante mas incompleto” em interrogações complexas. |

A terceira linha da tabela ilustra o comportamento de robustez do sistema: quando questionado sobre informação não presente no documento, o sistema recusa-se a inventar uma resposta, que é o comportamento esperado e desejável num sistema RAG.

### 4.2. Sumarização Automática

O módulo de sumarização permite gerar resumos em três extensões: curto (3–4 frases), médio (2–3 parágrafos) e longo (5–6 parágrafos detalhados). Antes de enviar o texto ao LLM, é aplicada uma função de pré-processamento (`_remove_references()`) que identifica e remove automaticamente secções de referências bibliográficas (“References”, “Referências”, “Bibliografia”) e declarações de uso de IA, usando expressões regulares. Esta filtragem evita que o LLM inclua listas de referências no resumo gerado, isto foi um problema observado nos testes iniciais.

### 4.3. Extração de Palavras-Chave

A extração de palavras-chave é realizada via *prompt engineering* estruturado: o LLM é instruído a responder exclusivamente com uma lista JSON de strings (e.g., `["machine learning", "RAG"]`,

"embeddings"])), sem preâmbulo nem formatação adicional. Um mecanismo de *fallback* baseado em expressões regulares tenta recuperar as keywords caso o JSON não seja válido. O número de palavras-chave é configurável entre 5 e 25 através de um slider na interface. Nos testes realizados, o sistema extraiu consistentemente 14–15 palavras-chave relevantes e representativas do conteúdo.

#### 4.4. Módulo de Tradução Automática

O módulo de tradução EN→PT é ativado opcionalmente pelo utilizador e aplicado como pós-processamento às respostas do RAG e aos resumos gerados. A implementação segmenta o texto em frases individuais, processa-as em batches de 5 frases com o modelo MarianMT, e combina os resultados finais. Este design em batch é mais eficiente do que traduzir frase a frase, reduzindo o overhead de inicialização do modelo por chamada.

#### 4.5. Interface Streamlit

A interface foi desenvolvida com Streamlit e organizada em quatro tabs funcionais: *Chat RAG*, *Resumo*, *Palavras-Chave* e *Exportar*. O sidebar permite configurar: o modelo LLM (com suporte a múltiplos modelos Ollama), o tamanho dos chunks, o parâmetro  $k$ , e ativar/desativar a tradução automática. Um indicador de estado verifica em tempo real se o servidor Ollama está acessível, alertando o utilizador caso contrário.

### 5. Avaliação e Testes

#### 5.1. Configuração de Hardware

Todos os testes foram realizados na seguinte configuração de hardware:

- **CPU:** AMD (processador portátil)
- **GPU:** NVIDIA GeForce RTX 3060 Laptop GPU (6GB VRAM)
- **RAM:** 16GB
- **Sistema Operativo:** Windows 11

O Ollama utilizou automaticamente a GPU NVIDIA via CUDA para inferência, tanto para o LLM como para o modelo de embeddings.

#### 5.2. Documentos de Teste

O sistema foi avaliado com dois documentos distintos:

- **Documento A:** Artigo académico em português sobre o NotebookLM da Google (8 páginas, ~3500 palavras), produzindo 8 chunks com os parâmetros padrão.
- **Documento B:** Artigo de investigação recente em inglês sobre SARDI [7], um framework de RAG dinâmico para modelos de linguagem de difusão (15 páginas, ~7000 palavras), produzindo 16 chunks.

#### 5.3. Tempos de Execução

A Tabela 3 apresenta os tempos de execução medidos para as operações principais do sistema, usando o Documento A como referência.

Os tempos de indexação são dominados pela geração de embeddings para cada chunk. Os tempos de resposta RAG são dominados pela inferência do LLM. Ambas as operações beneficiam da aceleração GPU via CUDA.

**Table 3**

Tempos de execução medidos no hardware de teste (RTX 3060 Laptop, 6GB VRAM)

| Operação                        | Tempo médio |
|---------------------------------|-------------|
| Extração de texto + chunking    | <1s         |
| Indexação vetorial (8 chunks)   | ~10s        |
| Resposta RAG (pergunta factual) | ~10s        |
| Geração de resumo médio         | ~7s         |
| Extração de 15 palavras-chave   | ~8s         |
| Geração de relatório PDF        | <1s         |

#### 5.4. Qualidade das Respostas

Os testes cobriram quatro dimensões de qualidade:

**Precisão factual:** O sistema respondeu corretamente a perguntas factuais específicas. Com  $k = 6$ , identificou corretamente o modelo Gemini 1.5 Pro e as duas inovações estruturais (MoE e janela de 2M tokens). Com  $k = 4$ , a resposta à primeira pergunta foi menos precisa, omitindo o nome do modelo.

**Robustez anti-alucinação:** Quando questionado sobre informação não presente no documento (preço de subscrição do NotebookLM), o sistema respondeu que não encontrou informação suficiente, sem inventar valores. Este é o comportamento correto e esperado de um sistema RAG bem configurado.

**Coerência conversacional:** O sistema demonstrou capacidade de responder a perguntas de seguimento sem perder o fio condutor da conversa. A pergunta “E quais são as suas limitações?”, feita imediatamente após uma pergunta sobre *source grounding*, foi corretamente interpretada como referindo-se às limitações do *source grounding* e não do sistema em geral.

**Tradução automática:** Com o Documento B (inglês) e a tradução ativada, as respostas foram geradas em português, confirmando o funcionamento do módulo MarianMT. Sem a tradução ativada, o modelo 11ama3.2:3b respondeu igualmente em português, graças às instruções explícitas no prompt, o módulo de tradução funciona assim como camada de segurança adicional.

## 6. Conclusão

O LocaLLM Docs demonstra que é possível construir um sistema RAG completo, funcional e de qualidade utilizando exclusivamente recursos de código aberto configurados localmente. A arquitetura de nove ficheiros facilita a manutenção e extensão do sistema, e a interface Streamlit torna-o acessível sem necessidade de conhecimentos técnicos. Com tempos de resposta inferiores a 15 segundos numa GPU de consumidor corrente, o sistema é suficientemente ágil para uso académico e profissional.

Os resultados dos testes confirmam que o sistema é preciso, robusto face a perguntas sem resposta no documento, e capaz de manter coerência conversacional ao longo de múltiplas trocas. A filtragem automática de referências bibliográficas nos resumos e a limitação do histórico de conversa no prompt são dois contributos de engenharia que melhoraram visivelmente a qualidade dos outputs.

Como trabalho futuro, identificam-se as seguintes direções: suporte a múltiplos documentos em simultâneo com fusão de índices ChromaDB, implementação de re-ranking dos chunks recuperados (e.g., com um modelo cross-encoder), suporte a OCR para PDFs digitalizados, e integração de modelos de maior dimensão como o 11ama3.1:8b para melhorar a qualidade das respostas em documentos muito técnicos.

## Acknowledgments

Durante a preparação deste trabalho, foi utilizado um modelo de linguagem (Claude) como apoio ao desenvolvimento do código, estruturação do relatório e revisão gramatical. Após a geração assistida do conteúdo, foi feita uma revisão manual, edição e formatação académica em  $\text{\LaTeX}$ .

## References

- [1] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation for knowledge-intensive nlp tasks, *Advances in Neural Information Processing Systems* 33 (2020) 9459–9474.
- [2] Ollama, Ollama: Run large language models locally, 2024. URL: <https://ollama.com>, acedido em junho de 2026.
- [3] Z. Nussbaum, J. X. Morris, B. Duderstadt, A. Mulyar, Nomic embed: Training a reproducible long context text embedder, 2024. URL: <https://arxiv.org/abs/2402.01613>.
- [4] Chroma, Chromadb: The ai-native open-source embedding database, 2024. URL: <https://www.trychroma.com>, acedido em junho de 2026.
- [5] Helsinki-NLP, Opus-mt: Open neural machine translation models, 2020. URL: <https://huggingface.co/Helsinki-NLP/opus-mt-en-pt>, acedido em junho de 2026.
- [6] L. Cimon, et al., fpdf2: Simple pdf generation for python, 2024. URL: <https://github.com/py-pdf/fpdf2>, acedido em junho de 2026.
- [7] P. Junger, J. Lovelace, L. Zhao, D. Go, K. Q. Weinberger, Self-augmenting retrieval for diffusion language models, 2026. URL: <https://arxiv.org/abs/2606.06474>, arXiv:2606.06474.